

Path-Coloring Algorithms for Plane Graphs

Aven Bross

Department of Computer Science

University of Alaska

Fairbanks, AK 99775-6670

dabross@alaska.edu

Glenn G. Chappell

Department of Computer Science

University of Alaska

Fairbanks, AK 99775-6670

chappellg@member.ams.org

Chris Hartman

Department of Computer Science

University of Alaska

Fairbanks, AK 99775-6670

cmhartman@alaska.edu

February 20, 2025

2010 Mathematics Subject Classification. Primary 05C38; Secondary 05C10, 05C15.

Key words and phrases. Path coloring, list coloring, algorithm.

Abstract

A path coloring of a graph G is a vertex coloring of G such that each color class induces a disjoint union of paths. We present two efficient algorithms to construct a path coloring of a plane graph.

The first algorithm, based on a proof of Poh, is given a plane graph, and produces a path coloring of the given graph using three colors.

The second algorithm, based on similar proofs by Hartman and Škrekovski, performs a list-coloring generalization of the above. The algorithm is given a plane graph and an assignment of lists of three colors to each vertex, and it produces a path coloring of the given graph in which each vertex receives a color from its list.

Implementations are available for all algorithms that are described.

1 Introduction

All graphs will be finite, simple, and undirected. See West [11] for graph theoretic terms.

A *path coloring* of a graph G is a vertex coloring (not necessarily proper) of G such that each color class induces a disjoint union of paths. A graph G is *path k -colorable* if G admits a path coloring using k colors.

Broere & Mynhardt conjectured [2, Conj. 16] that every planar graph is path 3-colorable. This was proven independently by Poh [8, Thm. 2] and by Goddard [6, Thm. 1].

Theorem 1.1 (Poh 1990, Goddard 1991). *If G is a planar graph, then G is path 3-colorable.* $\square$

It is easily shown that the “3” in Theorem 1.1 is best possible. In particular, Chartrand & Kronk [5, Section 3] gave an example of a planar graph whose vertex set cannot be partitioned into two subsets, each inducing a forest.

Hartman [7, Thm. 4.1] proved a list-coloring generalization of Theorem 1.1 (see also Chappell & Hartman [4, Thm. 2.1]). A graph G is *path k -choosable* if, whenever each vertex of G is assigned a list of k colors, there exists a path coloring of G in which each vertex receives a color from its list.

Theorem 1.2 (Hartman 1997). *If G is a planar graph, then G is path 3-choosable.* $\square$

Essentially the same technique was used by Škrekovski [10, Thm. 2.2b] to prove a result slightly weaker than Theorem 1.2.

We discuss two efficient path coloring algorithms based on proofs of the above theorems. We distinguish between a *planar* graph—one that can be drawn in the plane without crossing edges—and a *plane* graph—a graph with a given embedding in the plane.

In Section 2 we discuss notation and terminology.

In Section 3 we outline our graph representations and the basis for our computations of time complexity.

Section 4 covers an algorithm based on Poh’s proof of Theorem 1.1. The algorithm is given a plane graph; it produces a path coloring of the given graph using three colors.

Section 5 covers an algorithm based Hartman’s proof of Theorem 1.2, along with the proof of Škrekovski mentioned above. The algorithm is given a plane graph and an assignment of a list of three colors to each vertex; it produces a path coloring of the given graph in which each vertex receives a color from its list.

Section 6 provides benchmark results for implementations of each algorithm [3]. The section also discusses how each algorithm may be modified to benefit from parallelism.

2 Notation and Terminology

Let G be a plane graph with n edges and m vertices. All subgraphs of G will be considered plane graphs with the inherited embedding. We say G is *triangulated* if every face is a 3-cycle, and *weakly triangulated* if every face besides the outer face is a 3-cycle. By Euler’s formula $m \leq 3n - 6$; if G is triangulated, then $m = 3n - 6$.

If $v \in G$ and $u \in N(v)$, we define $\text{next}_v(u)$ to be the neighbor of v immediately counter-clockwise from u around v , and $\text{prev}_v(u)$ to be the neighbor immediately clockwise.

Let C be a cycle in a G . If $u, v \in C$, then we define $C[u, v]$ to be the clockwise u, v -path around C ; if $u = v$, then $C[u, u] = u$. Define $\text{Int}(C)$ to be the subgraph of G consisting of

C and all vertices and edges interior to C ; equivalently, $\text{Int}(C)$ is the maximal subgraph of G with outer face C .

Let P be a path in G . If $u, v \in P$, then we define $P[u, v]$ to be the u, v -path in P . We define $\text{Int}(P) = P$.

If H is a subgraph of G , let $N(H) = \bigcup_{v \in V(H)} N(v) \setminus V(H)$ be the set of vertices in $G - H$ with a neighbor in H .

3 Graph Representations and Time Complexity

We will represent a graph via *adjacency lists*: a list, for each vertex v , of the neighbors of v . A vertex is represented by an integer $0 \dots n - 1$, where $n = n(G)$ is the graph order.

A plane graph will be specified via a *rotation scheme*: a circular ordering, for each vertex v , of the edges incident with v , in the order they appear around v in the plane embedding; this completely specifies the combinatorial embedding of the graph. Rotation schemes are convenient when we represent a graph using adjacency lists; we simply order the adjacency list for each vertex v in counter-clockwise order around v ; no additional data structures are required.

The input for each algorithm will be a biconnected, weakly triangulated plane graph with n vertices and m edges, represented via adjacency lists. The input size will be n , the number of vertices. Note that $\mathcal{O}(m) = \mathcal{O}(n)$, so it is equivalent to take the input size to be m , the number of edges. Moreover, arbitrary simple planar graphs may be plane embedded and triangulated in $\mathcal{O}(n)$ time, see Boyer and Myrvold [1].

In Section 4, given an edge uv , we will need a constant time operation to find v 's entry in u 's adjacency list from u 's entry in v 's list. We define an *augmented adjacency list* to be an adjacency list such that for every edge uv a reference to v 's entry in u 's list is stored in u 's entry in v 's list, and vice versa. Given an adjacency list representation of a graph, an augmented adjacency list representation may be constructed in $\mathcal{O}(m)$ time via the following procedure.

Algorithm 3.1.

Input. An adjacency list representation Adj of a graph G .

Output. An augmented adjacency list representation AugAdj of G with the same rotation scheme as Adj .

Step 1. Construct an augmented adjacency list representation AugAdj with the same rotation scheme as Adj , leaving the reference portion of each entry uninitialized.

Step 2. For each vertex v construct an array $\text{Wrk}[v]$ of vertex-reference pairs with length $\deg(v)$. For each v from 0 to $n - 1$ iterate through $\text{AugAdj}[v]$. For each neighbor u in $\text{AugAdj}[v]$ append the pair $(v, r_v(u))$ to $\text{Wrk}[u]$, where $r_v(u)$ is a reference to u 's entry in $\text{AugAdj}[v]$.

Step 3. For each v from $n - 1$ to 0 iterate through $\text{Wrk}[v]$. Upon reaching a pair $(u, r_u(v))$ in $\text{Wrk}[v]$ the last element of $\text{Wrk}[u]$ will be $(v, r_v(u))$; see the paragraph below. Use $r_u(v)$ and $r_v(u)$ to look up and assign references for the edge uv in $\text{AugAdj}[u]$ and $\text{AugAdj}[v]$. Remove $(v, r_v(u))$ from the back of $\text{Wrk}[u]$.

After completing Step 2 in Algorithm 3.1 the array $\text{Wrk}[v]$ will contain a pair $(u, r_u(v))$ for each neighbor u of v , sorted in increasing order by the neighbor u . Suppose that v is the current vertex at a given iteration of Step 3 in Algorithm 3.1. For each edge $uw \in E(G)$ such that $u < w$ and $v < w$, prior iterations of Step 3 will have initialized the references for uw in $\text{AugAdj}[u]$ and $\text{AugAdj}[w]$, and also removed the pair $(w, r_w(u))$ from $\text{Wrk}[u]$. Therefore for each $(u, r_u(v))$ in $\text{Wrk}[v]$, the array $\text{Wrk}[u]$ will contain only entries for vertices w where $w \leq v$. Since $\text{Wrk}[u]$ is sorted in increasing order by the neighboring vertices, the last element of $\text{Wrk}[u]$ must be $(v, r_v(u))$.

4 Path Coloring

In this section we describe a linear time algorithm to path 3-color plane graphs. Let's first recount Poh's path 3-coloring strategy.

Lemma 4.1 (Poh 1990). *Let G be a biconnected, weakly triangulated plane graph with outer face C . Let $c : V(C) \rightarrow S \subsetneq \{1, 2, 3\}$ be a 2-coloring of C such that each color class induces a nonempty path. There exists an extension of c to a path 3-coloring $c : V(G) \rightarrow \{1, 2, 3\}$ such that for each $v \in G - C$, if $vu \in E(G)$ with $u \in C$, then $c(v) \neq c(u)$.*

Proof. If $n(G) \leq 3$, then $G = C$ and the path 2-coloring of C is a path 3-coloring of G . We proceed by induction on the order of G .

Let P_1, P_2 be the two paths induced by the 2-coloring of the outer face C . We may write $P_1 = C[u_1, w_1]$ and $P_2 = C[w_2, u_2]$ for vertices $u_1, u_2, w_1, w_2 \in C$.

Suppose C is an induced subgraph of G . Let $u, w \in V(G) - V(C)$ be the vertices such that u, u_2, u_1 and w, w_1, w_2 are faces of G ; note that u and w are uniquely determined, but it may be that $u = w$. Since C is an induced cycle in G and $G \neq C$, $G - C$ is connected. Let P_3 be a u, w -path in $G - C$ of minimal length, and note that P_3 is an induced subgraph of $G - C$.

Color each vertex of P_3 with the color in $\{1, 2, 3\} - S$ not used in the 2-coloring of C . Let $C_1 = P_1 + P_3 + uu_1 + ww_1$ and $C_2 = P_2 + P_3 + uu_2 + ww_2$. The subgraphs $\text{Int}(C_1)$ and $\text{Int}(C_2)$ together with the coloring c each satisfy the requirements of the lemma. By the inductive hypothesis there exist extensions of c to a path 3-coloring of $\text{Int}(C_1)$ and a path 3-coloring of $\text{Int}(C_2)$. Since $\text{Int}(C_1)$ and $\text{Int}(C_2)$ only share the vertices in P_3 on their respective outer faces, the colorings agree and form a path 3-coloring of G .

Suppose C is not an induced subgraph. Then there exists an edge $v_1v_2 \in E(G) - E(C)$ with $v_1 \in P_1$ and $v_2 \in P_2$. Let $C_1 = C[v_2, v_1] + v_1v_2$ and $C_2 = C[v_1, v_2] + v_1v_2$. By the inductive hypothesis, there exists an extension of c to a path 3-coloring of $\text{Int}(C_1)$ and $\text{Int}(C_2)$. Again the subgraphs only share vertices on their outer faces, and thus the colorings agree and combine form a path 3-coloring of G . $\square$

Let G be a triangulated plane graph. We may trivially path 2-color the outer triangle. By Lemma 4.1 this extends to a path 3-coloring of G . If G is an arbitrary planar graph, we may compute an embedding and add edges to produce a triangulated plane graph G' . Theorem 1.1 follows immediately as any path coloring of G' is also a path coloring of G .

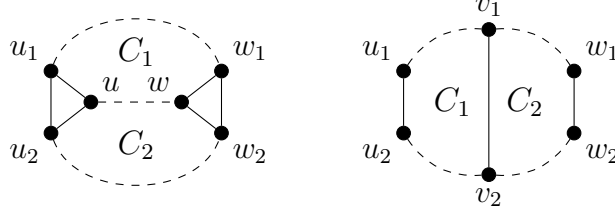


Figure 1: The proof of Lemma 4.1 when C is induced (left) or has a chord (right).

A natural way to implement Poh's algorithm is to locate u , and then use a breadth-first search to construct a minimal u, w -path and/or locate a chord edge, see Algorithm 4.2.

Algorithm 4.2.

Input. Let C be a cycle or length 2 path in a weakly triangulated plane graph G . Let Adj be an adjacency list representation of G . Let c be a length n array representing a 2-coloring of C such that each color class induces a path, respectively $P_1 = C[u_1, w_1]$ and $P_2 = C[w_2, u_2]$ for $u_1, u_2, w_1, w_2 \in C$. Assume $c[v] = 0$ for each $v \in \text{Int}(C) - C$.

The concrete input will be Adj , c , and the tuple $(u_1, w_1, u_2, w_2, u_1u_2)$, where u_1u_2 is represented by a reference to u_2 's entry in $\text{Adj}[u_1]$.

Output. For each vertex $v \in \text{Int}(C) - C$, a color in $\{1, 2, 3\}$ will be assigned to $c[v]$ such that c represents a path 3-coloring of $\text{Int}(C)$ extending the original 2-coloring of C . Moreover, if $v \in \text{Int}(C) - C$ has a neighbor $u \in C$, then $c[v] \neq c[u]$.

Base case. If $u_1 = w_1$ and $u_2 = w_2$, then C is a length 2-path and $\text{Int}(C) = C$ is path 2-colored.

Recursive case. Let $u \leftarrow \text{next}_{u_1}(u_2)$. Note that u, u_2, u_1 is a face of G .

Case 1. Suppose that $c[u] \neq 0$. If $c[u] = c[u_2]$, then $u \in P_2$. Make a recursive call with the input (u_1, w_1, u, w_2, u_1u) to color $\text{Int}(C[u_1, u] + u_1u)$. Otherwise, $c[u] = c[v_1]$ and $u \in P_1$. Iterate through $\text{Adj}[u]$ to find the entry for u_2 . Make a recursive call with the input (u, w_1, u_2, w_2, uu_2) to color $\text{Int}(C[u, u_2] + uu_2)$.

Case 2. Suppose that $c[u] = 0$, i.e. $u \in \text{Int}(C) - C$. Perform a breadth-first search in $\text{Int}(C) - C$ starting from u ; such a search may be implemented by visiting $v \in G$ only if $c[v] = 0$. Terminate the search upon finding a vertex w with a neighbor v_2 with $c[v_2] = c[u_2]$ and $c[\text{next}_w(v_2)] = c[u_1]$, i.e. $v_2 \in P_2$ and $v_1 = \text{next}_w(v_2) \in P_1$. Backtrack along the search tree from w to find a minimal u, w -path P_3 . For each vertex $v \in P_3$, assign $c[v] \leftarrow c(P_3)$, where $c(P_3) \in \{1, 2, 3\} - \{c[u_1], c[u_2]\}$.

Iterate through $\text{Adj}[u]$ to find uu_2 , i.e. the entry for u_2 . Similarly, iterate through $\text{Adj}[v_1]$ to find v_1v_2 . To color the remaining vertices we make three recursive calls:

1. with input (u_1, v_1, u, w, u_1u) to color $\text{Int}(C_1)$, where $C_1 = C[u_1, v_1] + P_3 + v_1u + v_1w$;
2. with input (u, w, u_2, v_2, uu_2) to color $\text{Int}(C_2)$, where $C_2 = C[v_2, u_2] + P_3 + uu_2 + uv_2$;
3. with input $(v_1, w_1, v_2, w_2, v_1v_2)$ to color $\text{Int}(C_3)$, where $C_3 = C[v_1, v_2] + v_1v_2$.

All remaining vertices in $\text{Int}(C) - C - P_3$ are now colored and we are done.

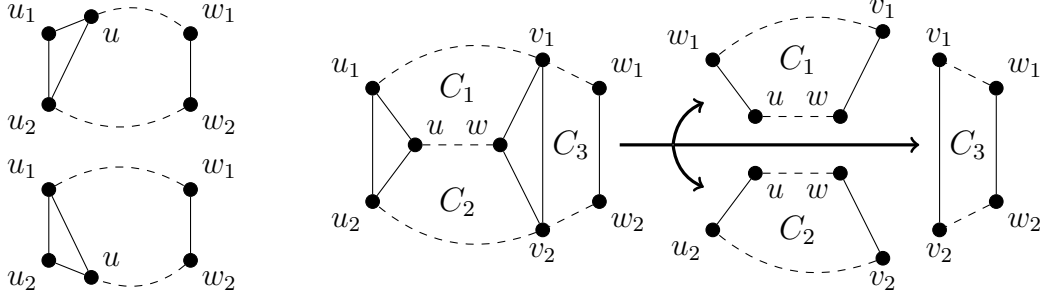


Figure 2: Algorithm 4.2, Case 1 (left) and Case 2 (right).

Let G be a triangulated plane graph. Let Adj be an adjacency list representation of G , and let $C = v_1, v_2, v_3$ be the outer triangle of G labeled in clockwise order. We may compute a path 3-coloring c of G as follows. Create a length n integer array c and initialize each entry to zero. Assign $c[v_1] \leftarrow 1$, $c[v_2] \leftarrow 2$, and $c[v_3] \leftarrow 2$. Finally, apply Algorithm 4.2 with input Adj , c , and $(v_1, v_1, v_2, v_3, v_1 v_3)$.

Unfortunately, Algorithm 4.2 is not linear. Consider the sequence of “pyramid graphs” $\{A_k\}_{k \in \mathbb{N}}$ depicted in Figure 3. Fix $k \in \mathbb{N}$ and note that $n = n(A_k) = k(k+1)/2 + 3$. Assume that the outer triangle is 2-colored such that the top vertex is assigned a color distinct from the bottom two. At recursive depth i , Algorithm 4.2 will perform a breadth first search of the pyramid-shaped interior subgraph with sides of length $r = k-i$, starting from the vertex in the bottom left corner. The search will hit all $r(r+1)/2$ vertices of the interior pyramid. Therefore the total number of operations performed will be

$$\Omega \left(\sum_{r=1}^k \frac{r(r+1)}{2} \right) = \Omega(k^3) = \Omega(n^{3/2}).$$

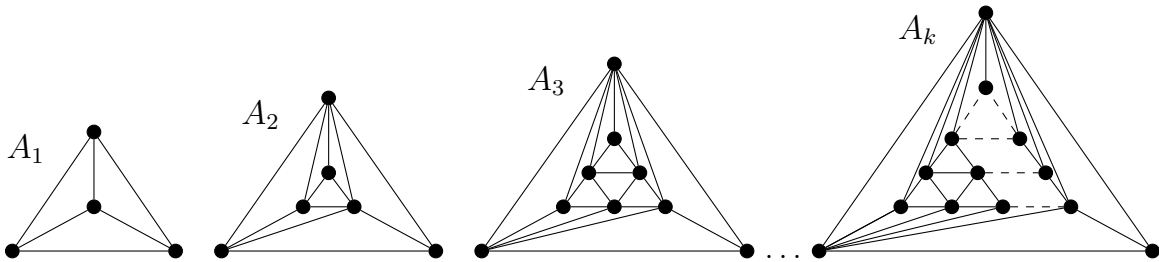


Figure 3: The sequence of “pyramid graphs” on which Algorithm 4.2 performs poorly.

The correctness of Poh’s proof, however, relied only on locating an induced u, w -path. Algorithm 4.3 describes a linear time implementation of Poh’s algorithm that locates and colors an induced u, w -path that is not always of minimal length.

For $P_1 = C[u_1, w_1]$ let $p_1 : N(P_1) \rightarrow V(P_1)$ to be the function that maps each vertex to its furthest clockwise neighbor along P_1 , i.e. $V(P_1[p_1(v), w_1]) \cap N(v) = \{p_1(v)\}$.

Algorithm 4.3.

Input. Let C be an induced cycle in a weakly triangulated plane graph G . Let Adj be an adjacency list representation of G . Let c be a length n array of colors representing a 2-coloring of C such that each color class induces a path, respectively $P_1 = C[u_1, w_1]$ and $P_2 = C[w_2, u_2]$ for $u_1, u_2, w_1, w_2 \in C$. Assume $c[v] = 0$ for each $v \in \text{Int}(C) - C$. Let S be a length n array of integer marks and let $m(P_1)$ be an integer such that for each $v \in \text{Int}(C) - C$ we have $S[v] = m(P_1)$ if and only if $v \in N(P_1)$.

The concrete input will be the arrays Adj , c and S , along with the ordered pair $(u_1, u_1u_2, m(P_1))$ where u_1u_2 is represented by the entry for u_2 in $\text{Adj}[u_1]$.

Output. For each vertex $v \in \text{Int}(C) - C$, a color in $\{1, 2, 3\}$ will be assigned to $c[v]$ such that c represents a path 3-coloring of $\text{Int}(C)$ extending the original 2-coloring of C . Moreover, if $v \in N(P_1)$, then $c[v] \neq c[u_1]$, and if $v \in N(P_2)$, then $c[v] \neq c[u_2]$.

Procedure. Assign $u \leftarrow \text{next}_{u_1}(u_2)$.

Base case. If $c[u] \neq 0$, then $u = w_2$ or $u = w_1$. Thus C is a face and we are done.

Recursive case. Suppose that $c[u] = 0$, i.e. $u \in \text{Int}(C) - C$.

Step 1. Assign $c[u]$ the remaining color in $\{1, 2, 3\} \setminus \{c[u_1], c[u_2]\}$. Let $m(P_3)$ be a new unique mark. We will iteratively construct an induced u, w -path $P_3 = v_1, v_2, \dots, v_j$ with vertices $N(P_1)$, starting from $v_1 = u$, and color it with $c[u]$. We will simultaneously mark the vertices in $N(P_3) - V(C)$ with $m(P_3)$, and recursively color $\text{Int}(C) - C - P_3$.

Assign $v_0 \leftarrow u_2$, $v_1 \leftarrow u$, and $j \leftarrow 1$. Assign $i \leftarrow 1$ and $v_i s \leftarrow uu_2$, where $v_i s$ represents the entry for s in $\text{Adj}[v_i]$.

Step 2. Assign $c[v_j] \leftarrow c[u]$. We will iterate through the neighbors of v_j , starting after v_{j-1} . Assign $v \leftarrow \text{next}_{v_j}(v_{j-1})$ and $y \leftarrow \text{NULL}$. Also assign $v_j t \leftarrow \text{NULL}$, where $v_j t$ will represent an optional entry for a neighbor t in $\text{Adj}[v_j]$.

Step 3. There are several cases to consider.

Case 3.1. Suppose $c[v] = 0$ and $S[v] \neq m(P_1)$. Assign $S[v] \leftarrow m(P_3)$ and $v \leftarrow \text{next}_{v_j}(v)$. Go to Step 3.

Case 3.2. Suppose $c[v] = c[u_2]$, i.e. $v \in P_2$. Make a recursive call with input $(v_i, v_i s, m(P_3))$ to color $\text{Int}(P_3[v_i, v_j] + C[v, s] + v_i s + v_j v)$. Assign $i \leftarrow j$, $v_j s \leftarrow v_j v$, and $v \leftarrow \text{next}_{v_j}(v)$. Go to Step 3.

Case 3.3. Suppose that $c[v] = 0$ and $S[v] = m(P_1)$. If $y = \text{NULL}$, assign $y \leftarrow v$. Assign $S[v] \leftarrow m(P_3)$ and $v \leftarrow \text{next}_{v_j}(v)$. Go to Step 3.

Case 3.4. Suppose that $c[v] = c[u_1]$, i.e. $v = p_1(v_j) \in P_1$. Assign $v_j t \leftarrow v_j v$ and $v \leftarrow \text{next}_{v_j}(v)$. Go to Step 4.

Note that if $y = \text{NULL}$, then $v_j = w$. Since $v \in P_1$, $\text{prev}_{v_j}(v) \notin N(P_1)$, and $\text{prev}_{v_j}(v) \notin P$, we must have $\text{prev}_{v_j}(v) \in P_2$.

Step 4. We now consider the neighbors of v_j counter-clockwise past t .

Case 4.1. Suppose that $c[v] = 0$. Assign $S[v] \leftarrow m(P_3)$ and $v \leftarrow \text{next}_{v_j}(v)$. Go to Step 4.

Case 4.2. Suppose that $c[v] \neq 0$. Make a recursive call with input $(v_j, v_j t, m(P_3))$.

If $c[v] = c[u_1]$, i.e. $v \in P_1$, then the recursive call colors $\text{Int}(C[v, t] + v_j + v_j t + v_j v)$. Assign $v_j t \leftarrow v_j v$. Go to Step 4.

Otherwise, $v = v_{j-1}$ and the recursive call colors $\text{Int}(C[p_1(v), t] + P_3[v, v_j] + v p_1(v) + v_j t)$. Go to Step 5. (Note that we do not need to compute $p_1(v)$.)

Step 5. If $y \neq \text{NULL}$, assign $j \leftarrow j + 1$, $v_j \leftarrow y$, and go to Step 2. Otherwise, $v_j = w$ and we are done.

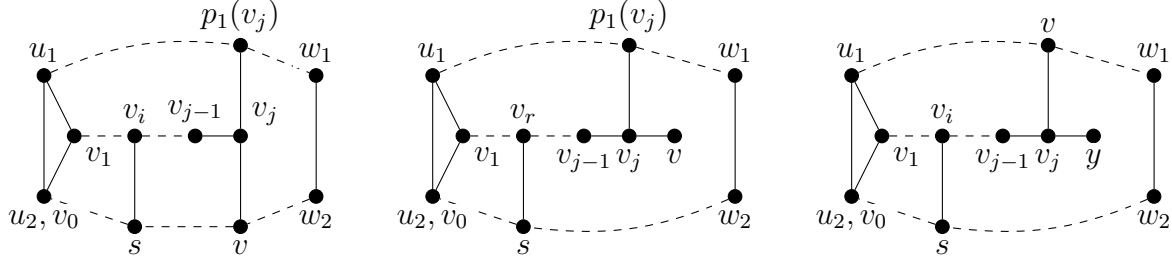


Figure 4: Algorithm 4.3, Case 3.2 (left), Case 3.3 (middle), and Case 3.4 (right).

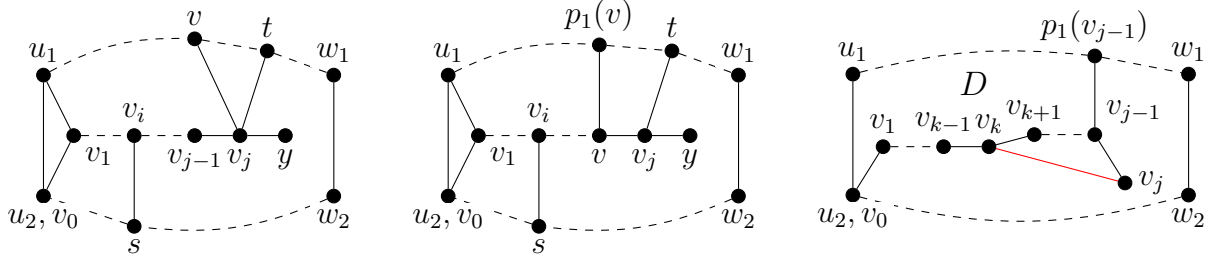


Figure 5: Algorithm 4.3, Case 4.2 when $v \in P_1$ (left) and $v \in P_3$ (middle). Also, a sketch of why P_3 is an induced path (right).

We claim that the path $P_3 = v_1, v_2, \dots, v_j$ constructed in Algorithm 4.3 is always an induced path in G . Suppose that $v_k v_j \in E(G)$ with $0 < k < j - 1$. Observe that

$$D = C[u_2, p_1(v_{j-1})] + P_3[v_1, v_{j-1}] + u_2 v_1 + v_{j-1} p_1(v_{j-1})$$

is a cycle in G with $v_{k-1}, v_k, v_{k+1} \in D$ and $v_j \notin \text{Int}(D)$, see Figure 5 (right). Therefore $v_k v_j \notin \text{Int}(D)$ and $v_k v_j$ lies between v_{k-1} and v_{k+1} counter-clockwise in $\text{Adj}[v_k]$. But v_{k+1} is the first neighbor in $N(P_1)$ counter-clockwise from v_{k-1} around v_k , a contradiction.

Given a triangulated plane graph G with adjacency list representation Adj , we can apply Algorithm 4.3 as follows. First, create length n integer arrays for c and S , initializing each entry to zero. Let $C = v_1, v_2, v_3$ be the outer triangle of G , labeled in clockwise order. Color $c[v_1] \leftarrow 1$, $c[v_2] \leftarrow 2$, and $c[v_3] \leftarrow 2$. Iterate through $\text{Adj}[v_1]$ and mark $S[v] \leftarrow 1$ for each neighbor v of v_1 . Let $u \leftarrow \text{next}_{v_1}(v_3)$. Note that if $n(G) > 3$, then $u \notin C$. The arrays Adj , c , and S , together with (u, uv_3) form a valid input for Algorithm 4.3.

While executing Algorithm 4.3 we iterate through the adjacency list of each vertex $v \in G$ exactly twice: once when v is colored and added to a path P_3 in Step 2 to locate the

entry for v_{j-1} , and once to examine each neighbor in $\text{Adj}[v_j]$ during Steps 3-4. Therefore the time complexity of the algorithm is $\mathcal{O}(m) = \mathcal{O}(n)$.

5 Path List-Coloring

In this section we describe a linear time algorithm to path color plane graphs such that each vertex receives a color from a specified list. Hartman showed that this is always possible when each vertex is given a list of 3 colors [7, Thm. 4.1]. Around the same time Škrekovski proved a weaker result using the same coloring strategy [10, Thm. 2.2b].

The path list-coloring procedure discussed in this section is based on the constructive proofs found in Hartman and Škrekovski's papers. For those familiar with Hartman's proof: if $x = z$, then our G, x, y match with those in [7, Lemma. 4.2], but our G is always biconnected; we follow Hartman's iterative procedure to color a path P clockwise from x along C , but we proceed inductively, one edge at a time, and simultaneously remove P from the subgraph in consideration. If $x \neq z$, then z is the furthest clockwise end of the path P being colored; the the rest of P (including the original " x ") has been removed.

A *list assignment* of a graph G is a function $L : V(G) \rightarrow P_{<\aleph_0}(\mathbb{N})$ assigning a finite set of colors to each vertex. If L is a list assignment of G , an *L -list-coloring* or *L -coloring* of G is a coloring function $c : V(G) \rightarrow \mathbb{N}$ such that $c(v) \in L(v)$ for each $v \in V(G)$.

Given a graph G and a coloring c of G , for each $v \in G$ we define $\deg_c(v)$ to be the number of neighbors of v that share a color with v . Equivalently, $\deg_c(v)$ is the degree of v in the subgraph of G induced by the color class of $c(v)$.

Lemma 5.1. *Let G be a biconnected, weakly triangulated plane graph with outer face C . Let $x, y, z \in C$ be vertices (not necessarily distinct) such that $z \in C[x, y]$. Let L be a list assignment of G such that*

$$\begin{aligned} |L(v)| &= 1 \text{ for } v \in \{x, y, z\}, \\ |L(v)| &\geq 2 \text{ for } v \in C - \{x, y, z\}, \\ |L(v)| &\geq 3 \text{ for } v \in G - C, \end{aligned}$$

and if $v \in C[x, z] - z$, then $L(v) \cap L(z) = \emptyset$. There exists a path L -coloring of G such that $\deg_c(x) \leq 1$, $\deg_c(y) \leq 1$, and $\deg_c(z) \leq 1$. Moreover, if $z = y$ or $C[z, y] = z, y$ and $L(z) \cap L(y) = \emptyset$, then $\deg_c(z) = 0$.

Proof. Suppose that $m = |E(G)| \leq 1$. Then $G = C$. We define a path L -coloring c of G as follows. Let $c(x) \in L(x)$, $c(y) \in L(y)$, and $c(z) \in L(z)$ be the colors in each respective list. If $m = 0$, then $n(G) = 1$ and c is trivially a path L -coloring. Otherwise $m = 1$ and $n(G) = 2$. If $x \neq y$, then c is a path L -coloring of G . Otherwise the remaining $v \in G - x$ satisfies $|L(v)| \geq 2$ and we define $c(v) \in L(v) \setminus \{c(x)\}$.

We proceed by induction on m , the number of edges. Let $u_1, u_2, \dots, u_k$ label the neighbors of z in counter-clockwise order such that u_1 is the vertex immediately counter-clockwise from z around C and u_k is the vertex clockwise from z around C . In each of

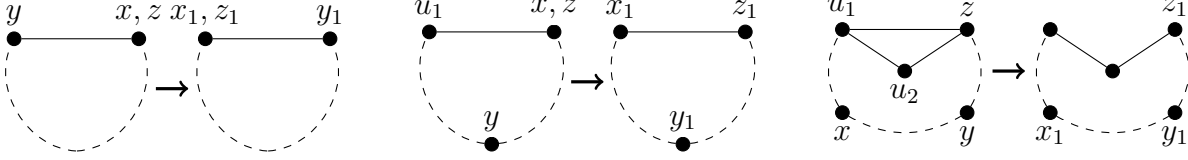


Figure 6: The proof of Lemma 5.1, Case 1 (left), Case 2 (middle), and Case 3.1 (right).

the subsequent cases we will apply the lemma separately to one or two subgraphs G_1, G_2 of G with corresponding labeled outer face vertices $x_i, y_i, z_i \in G_i$ for $i = 1, 2$.

Case 1. Suppose that $u_1 = y$. Note that $z = x$ since $z \in C[x, y]$. Therefore $V(C[z, y]) = V(C)$ and $|V(C[z, y])| \geq 3$. Apply the lemma to find a path L -coloring c of $G_1 = G$ with $x_1 = y, y_1 = x$, and $z_1 = y$ to .

Case 2. Suppose that $u_1 \neq y$ and $z = x$. Then $u_1 \in C - C[x, z]$ and $|L(u_1)| \geq 2$. Pick $d \in L(u_1) \setminus L(z)$. Define $L'(u_1) = \{d\}$ and $L'(v) = L(v)$ for each $v \in G - u_1$. Apply the lemma to find a path L' -coloring of $G_1 = G$ with $x_1 = u_1, y_1 = y$, and $z_1 = z$.

Case 3. Suppose that $u_1 \neq y$ and $z \neq x$. Our strategy will be to apply the inductive hypothesis to the blocks of $G - zu_1$. Because $u_1 \in C[x, z]$, it is guaranteed that $L(u_1) \cap L(z) = \emptyset$. Thus any path L -coloring of $G - zu_1$ is also a path L -coloring of G .

Case 3.1. Suppose that $u_2 \in G - C$. Define $L'(u_2) = L(u_2) \setminus L(z)$ and $L'(v) = L(v)$ for each $v \in G - u_2$. Note that $|L'(u_2)| \geq |L(v)| - 1 \geq 2$. Let C_1 be the outer face of $G_1 = G - zu_1$, and observe that $C_1[x, z]$ is equal to $C[x, z]$ with the edge u_1z removed and the path u_1, u_2, z added. Since $|L'(u_2)| \geq 2$ and $L'(u_2) \cap L'(z) = \emptyset$, we may apply the inductive hypothesis to find a path L' -coloring of G_1 with $x_1 = x, y_1 = y, z_1 = z$.

Case 3.2. Suppose that $u_2 \in C$. Observe that u_2 is a cut-vertex of $G - zu_1$. Define $C_1 = C[u_2, u_1] + u_1u_2$ and $C_2 = C[z, u_2] + zu_2$. Define $G_1 = \text{Int}(C_1)$ and $G_2 = \text{Int}(C_2)$. The subgraphs G_1 and G_2 are the two biconnected components (blocks) of $G - zu_1$. Note that if $k = 2$, then $G_1 = G - z$ and $G_2 = z, u_2$.

In each subsequent case we will apply the inductive hypothesis to find a path L -coloring c_1 of G_1 and a path L -coloring c_2 of G_2 such that $c_1(u_2) = c_2(u_2)$. Let c be the L -coloring of G defined by $v \mapsto c_1(v)$ for $v \in G_1$ and $v \mapsto c_2(v)$ for $v \in G_2$. Observe that $\deg_c(v) = \deg_{c_1}(v)$ for each $v \in G_1 - u_2$, $\deg_c(v) = \deg_{c_2}(v)$ for each $v \in G_2 - u_2$, and $\deg_c(u_2) = \deg_{c_1}(u_2) + \deg_{c_2}(u_2)$. To show that c is a path L -coloring of G it suffices to show that $\deg_c(u_2) \leq 2$.

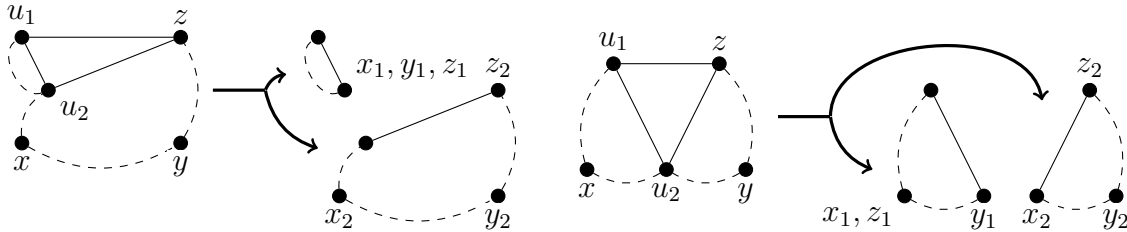


Figure 7: The proof of Lemma 5.1, Case 3.2.1 (left) and Case 3.2.2 (right).

Case 3.2.1. Suppose that $u_2 \in C[x, z]$. Observe that $x, y, z \in C_2$. Apply the inductive hypothesis to find a path L -coloring c_2 of G_2 with $x_2 = x$, $y_2 = y$, and $z_2 = z$. Define $L'(u_2) = \{c_2(u_2)\}$ and $L'(v) = L(v)$ for each $v \in G_1 - u_2$. Find a path L' -coloring c_1 of G_1 with $x_1 = y_1 = z_1 = u_2$. Since $\deg_{c_1}(u_2) = 0$, it follows that c is a path L -coloring of G satisfying the lemma.

Case 3.2.2. Suppose that $u_2 \in C[y, x] - x - y$. Pick $c(u_2) \in L(u_2) - c(z)$. Define $L'(u_2) = \{c(u_2)\}$ and $L'(v) = L(v)$ for each $v \in G - u_2$. Observe that $x \in C_1$ and $z, y \in C_2$. Apply the inductive hypothesis to find path L' -coloring c_1 of G_1 with $x_1 = z_1 = x$ and $y_1 = u_2$. Then find a path L' -coloring c_2 of G_2 with $x_2 = u_2$, $y_2 = y$, and $z_2 = z$. Note that $\deg_{c_1}(u_2) \leq 1$ and $\deg_{c_2}(u_2) \leq 1$, and thus c is a path L -coloring of G satisfying the lemma.

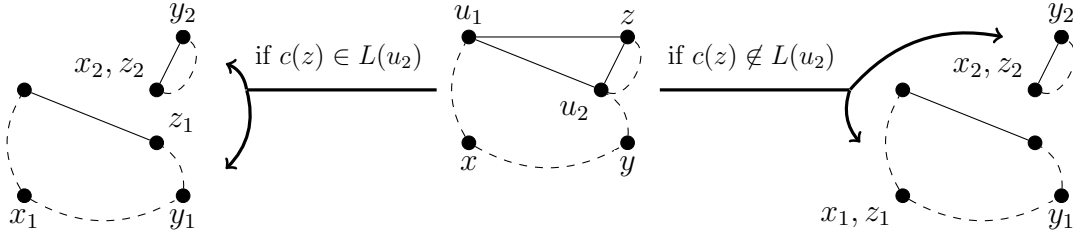


Figure 8: The proof of Lemma 5.1, Case 3.2.3.

Case 3.2.3. Suppose that $u_2 \in C[z, y]$. Note that $z \neq y$. There are two distinct cases to consider.

Case 3.2.3.1. Suppose that $c(z) \in L(u_2)$. Then either $u_2 = y$ and $L(z) \cap L(y) = \{c(z)\}$, or $C[z, y] \neq z, y$. Define $L'(u_2) = \{c(z)\}$ and $L'(v) = L(v)$ for each $v \in G - u_2$. Find a path L' -coloring c_1 of G_1 with $x_1 = x$, $y_1 = y$, and $z_1 = u_2$. Then find a path L' -coloring c_2 of G_2 with $x_2 = u_2$, $y_2 = z$, and $z_2 = u_2$. Since $\deg_{c_1}(u_2) \leq 1$ and $\deg_{c_2}(u_2) = 1$, it follows that c is a path L -coloring of G .

Case 3.2.3.2. Suppose that $c(z) \notin L(u_2)$. Find a path L -coloring c_1 of G_1 with $x_1 = x$, $y_1 = y$, and $z_1 = x$. Define $L'(u_2) = \{c_1(u_2)\}$ and $L'(v) = L(v)$ for each $v \in G_2 - u_2$. Then find a path L' -coloring c_2 of G_2 with $x_2 = u_2$, $y_2 = z$, and $z_2 = u_2$. Because $C[z_2, y_2] = z_2, y_2$ and $L'(z_2) \cap L'(y_2) = \emptyset$, it is guaranteed that $\deg_{c_2}(u_2) = \deg_{c_2}(z_2) = 0$. Thus c is a path L -coloring of G .

Suppose that $C[z, y] = z, y$. Then it must be that $u_2 = y$ and $k = 2$. Therefore $G_2 = C_2 = z, y$ and $\deg_c(z) = 0$ since $c(z) \notin L(u_2)$. $\square$

Let G be a planar graph and L a list assignment such that $|L(v)| \geq 3$ for each $v \in G$. Compute a planar embedding and add edges to produce a triangulated plane graph G' . Pick $u \in G'$ to be a vertex on the outer face and $d \in L(u)$. Define L' to be the list assignment such that $L'(u) = \{d\}$ and $L'(v) = L(v)$ for $v \in G' - u$. By Lemma 5.1 there exists a path L' -coloring c of G' . Clearly c is also a path L -coloring of G . Therefore Theorem 1.2 follows immediately from Lemma 5.1.

The proof of Lemma 5.1 is constructive and may be implemented as a linear time algorithm for plane graphs represented by rotation scheme ordered augmented adjacency

lists. We will assume that both forward and backward iteration in augmented adjacency lists is a constant time operation, i.e. given $u \in G$ and an entry for an edge uv in $\text{Adj}[u]$, $\text{next}_u(v)$ and $\text{prev}_u(v)$ can be computed in constant time. The available C implementation represents each augmented adjacency list as an array of entries, but e.g. a doubly linked list would also suffice [3].

Algorithm 5.2.

Input. Let C be a cycle or length 2 path in a weakly triangulated plane graph G . Let $x, y, z \in C$ be vertices such that $z \in C[x, y]$. Let Adj be an augmented adjacency list representation of G .

Let L be an array of lists of colors such that for $v \in \{x, y, z\}$ the list $L[v]$ has length one, for $v \in C - \{x, y, z\}$ the list $L[v]$ has length two or three, and for $v \in \text{Int}(C) - C$ the list $L[v]$ has length three. Furthermore, for each $v \in C[x, z] - z$ assume that $L[v]$ does not contain the color in $L[z]$.

Let N be an array of pairs of references such that for each $v \in C$ the pair $N[v] = (r_1, r_2)$ contains a reference r_1 to the $\text{Adj}[v]$ entry for neighbor the immediately counter-clockwise from v around C , and a reference r_2 to the entry for the neighbor immediately clockwise from v around C .

Let S be an array of integers such that if $v \in C$, then $S[v] \neq 0$, and if $v \in \text{Int}(C) - C$, then $S[v] = 0$. Also, if $v \in C$, then $S[v] = S[x]$ if and only if $v \in C[x, z] - z$. Let M be an array of integers such that if $v \in C$, then $M[S[v]] = S[y]$ if and only if $v \in C[z, y]$.

The concrete input will be Adj , L , N , S , M , and the tuple (x, y, z) .

Output. For each $v \in \text{Int}(C)$, all but one color will be removed from the list $L[v]$. The remaining color in each list will represent a path L -coloring of $\text{Int}(C)$ such that $\deg_c(x) \leq 1$, $\deg_c(y) \leq 1$, and $\deg_c(z) \leq 1$. If $z = y$, or if $C[z, y] = z, y$ and $L[z]$ did not contain the color in $L[y]$, then $\deg_c(z) = 0$.

Procedure. Let $N[z] = (r_1, r_2)$. Assign the vertex u_1 to be the neighbor of z corresponding to r_1 .

Base case. If $r_1 = r_2$, then $C = z, u_1$ is a length 2 path. If $u_1 \neq x$ and $u_1 \neq y$, remove the color in $L[z]$ from $L[u_1]$. If more than one color still remains in $L[u_1]$, remove arbitrary colors until a single color remains.

Recursive case. Suppose that $r_1 \neq r_2$. Note that $n(C) > 2$ and therefore C is a cycle. Assign $u_2 \leftarrow \text{next}_z(u_1)$.

Case 1. Suppose that $u_1 = y$. Then $z = x$, since $z \in C[x, y]$. Assign $S[x]$ and $S[y]$ the same new unique mark. Assign $M[S[x]] \leftarrow S[x]$. Make a recursive tail call with the input (y, x, y) .

Case 2. Suppose that $z = x$ and $u_1 \neq y$. Remove the color in $L[z]$ from $L[u_1]$, and then remove arbitrary colors from $L[u_1]$ until a single color remains. Set $S[u_1]$ to be a new unique mark, assign $M[S[u_1]] \leftarrow S[u_1]$, and make a recursive call with input (u_1, y, z) .

Case 3. Suppose that $z \neq x$ and $u_1 \neq y$. It is this case that makes use of the back references in the augmented adjacency list representation.

Case 3.1. Suppose that $S[u_2] = 0$, that is, $u_2 \in \text{Int}(C) - C$. Assign $S[u_2] \leftarrow S[x]$ and initialize $N[u_2] \leftarrow (s_1, s_2)$ where s_1 is a reference to the entry for u_1 in $\text{Adj}[u_2]$, and s_2 is

a reference to the entry for z . Remove the color in $L[z]$ from $L[u_2]$ and remove the edge zu_1 by adjusting $N[u_1]$ and $N[z]$. Make a recursive call with the same input (x, y, z) .

Case 3.2. Suppose that $S[u_2] \neq 0$, that is, $u_2 \in C$. Just as in Case 3.2 of the proof of Lemma 5.1, we will separately consider the two blocks G_1 and G_2 of $\text{Int}(C) - zu_1$.

It is simple to remove the edge zu_1 by adjusting $N[z]$ and $N[u_1]$ to exclude the corresponding entries. It remains to “split” the neighborhood of u_2 at the edge u_2z such that the recursive call on G_1 considers only the neighbors of u_2 counter-clockwise from z , and the call on G_2 considers z and all neighbors clockwise from z .

Let $N[u_2] = (s_1, s_2)$. Let s_3 be a reference to z ’s entry in $\text{Adj}[u_2]$ and s_4 a reference to $\text{next}_{u_2}(z)$ ’s entry. To represent the subgraph G_1 , assign $N[u_2] \leftarrow (s_4, s_2)$. To represent the subgraph G_2 , assign $N[u_2] \leftarrow (s_1, s_3)$.

Case 3.2.1. Suppose that $S[u_2] = S[x]$, i.e. $u_2 \in C[x, z]$. Make a recursive call on G_2 with input (x, y, z) . Assign $S[u_2]$ a new unique mark, set $M[S[u_2]] \leftarrow S[u_2]$, and make a recursive call on G_1 with input (u_2, u_2, u_2) .

Case 3.2.2. Suppose that $z \neq x$ and $S[u_2] \neq 0$, but $S[u_2] \neq S[x]$ and $M[S[u_2]] \neq S[y]$. In other words, suppose that $u_2 \in C[y, x] - x - y$. First remove the color in $L[z]$ from $L[u_2]$, if it exists, and then remove arbitrary colors until $L[u_2]$ is of length one. Assign $S[u_2] = S[x]$. Make a recursive call on G_1 with input (x, u_2, x) . Assign $S[u_2]$ a new unique mark and $M[S[u_2]] \leftarrow S[u_2]$. Make a recursive call on G_2 with input (u_2, y, z) .

Case 3.2.3. Suppose that $z \neq x$ and $M[S[u_2]] = S[y]$, i.e. $u_2 \in C[z, y]$. There are two cases to consider.

Case 3.2.3.1. Suppose that the color in $L[z]$ is in $L[u_2]$. Remove all colors from $L[u_2]$ except for the color in $L[z]$. Make a recursive call on G_1 with input (x, y, u_2) . Then assign $S[z]$ and $S[u_2]$ the same new unique mark, and assign $M[S[z]] \leftarrow S[z]$. Make a recursive call on G_2 with input (u_2, z, u_2) .

Case 3.2.3.2. Suppose that the color in $L[z]$ is not in $L[u_2]$. Assign $M[S[x]] \leftarrow S[y]$. Make a recursive call on G_1 with input (x, y, x) . Assign $S[z]$ and $S[u_2]$ the same new unique mark, and set $M[S[z]] \leftarrow S[z]$. Make a recursive call on G_2 with input (u_2, z, u_2) .

Note that this is the only case where $M[S[v]]$ is set a value other than $S[v]$. Setting $M[S[x]] \leftarrow S[y]$ effectively “joins” $C[x, u_1]$ and $C[u_2, y]$ into $C_1[z_1, y_1]$ in G_1 , see Figure 8.

Let G be a triangulated plane graph represented by augmented adjacency lists. Let L be a size n array of color lists of length 3, representing a list assignment of G . We may path L -color G using Algorithm 5.2 as follows.

Create an array of reference pairs N of size n , and initialize $N[v]$ for each $v \in C$ on the outer face of G to satisfy the requirements of Algorithm 5.2. Create a size n array of integers S , and initialize $S[v] \leftarrow 1$ for each $v \in C$ and $S[v] \leftarrow 0$ for each $v \in G - C$. Pick an arbitrary $u \in C$, assign $S[u] \leftarrow 2$, and remove all but one color from $L[u]$. Create a size $m + 1$ array of integers M and initialize $M[i] \leftarrow i$ for each $i \in \{0, 1, 2\}$. Finally, apply Algorithm 5.2 with the input Adj , L , N , S , M , and (u, u, u) . The remaining color in each list in L will represent a path L -coloring of G .

Every recursive case in Algorithm 5.2 performs a fixed number of constant time operations. Each case removes one edge from the graph representation, except for Case 1 and Case 2. In Case 1 a single recursive call is made with an input that will not itself satisfy

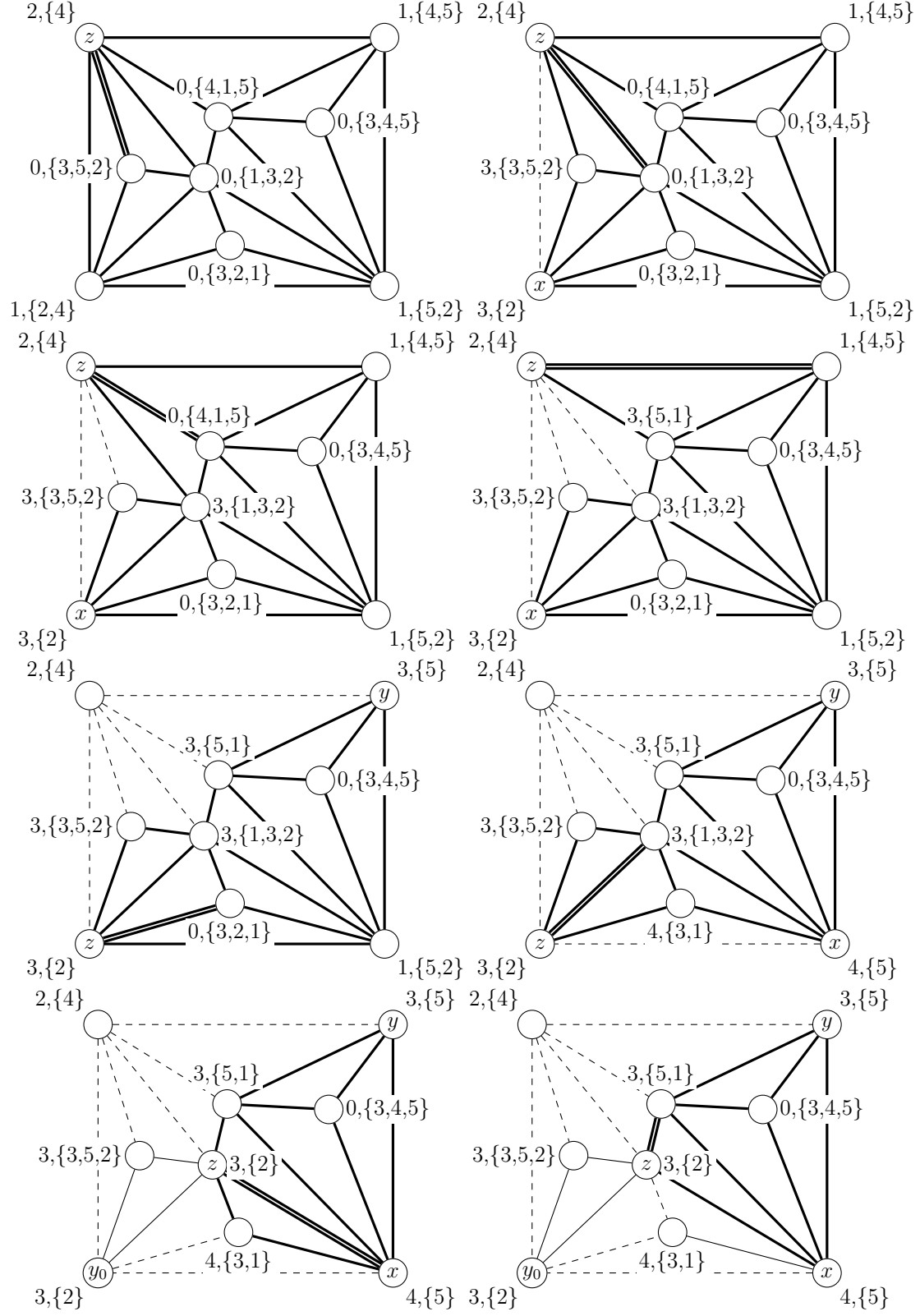


Figure 9: An example of Algorithm 5.2. Each vertex $v \in G$ is labeled with $S[v], L[v]$.

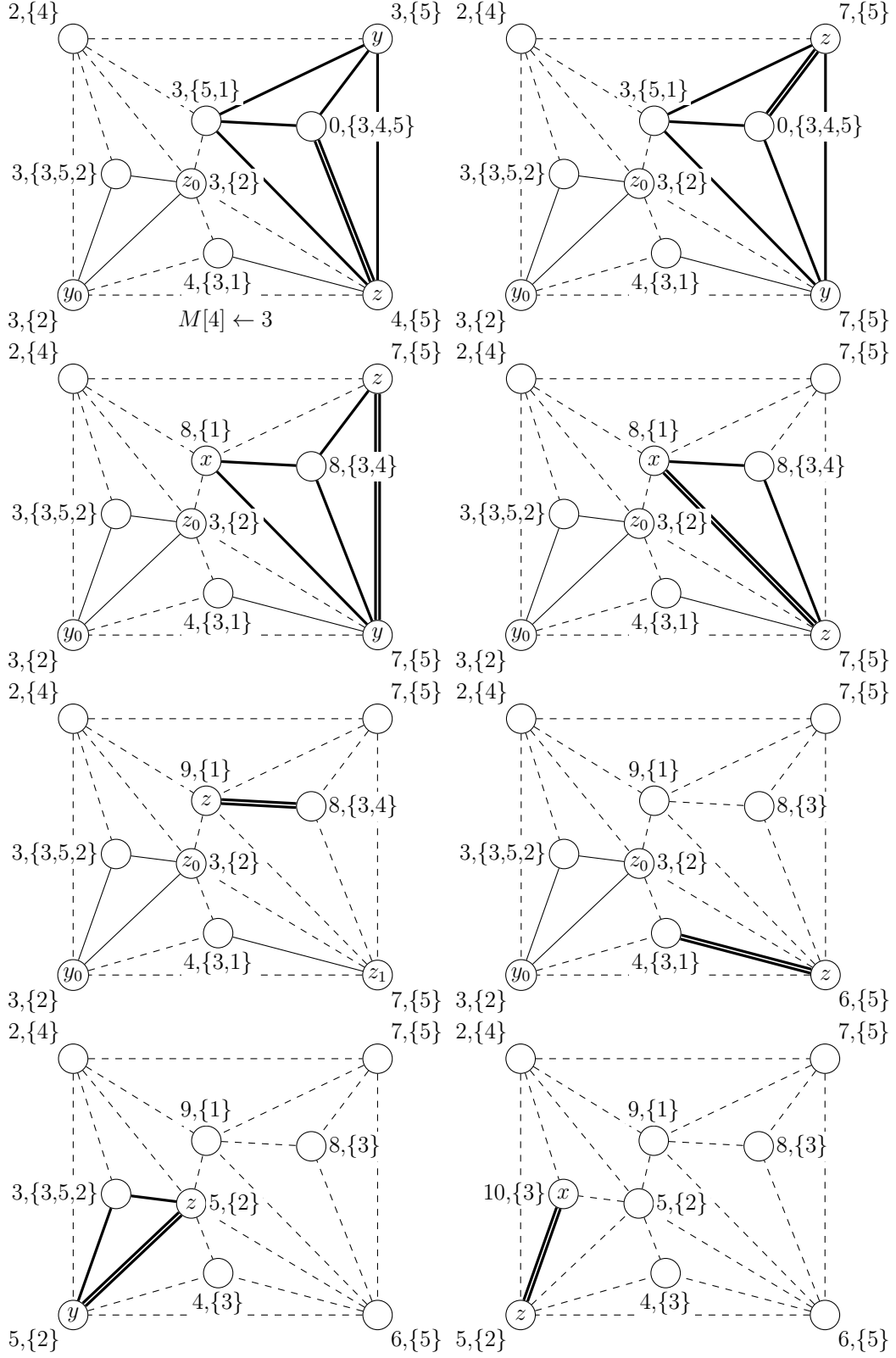


Figure 10: An Algorithm 5.2 example, continued from Figure 9.

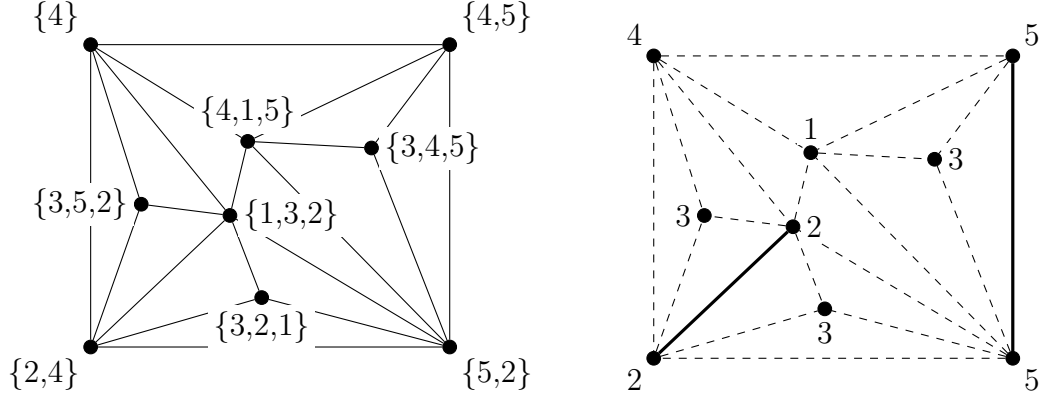


Figure 11: The initial list assignment L (left) and the induced paths of the resulting path L -coloring (right) from the Algorithm 5.2 example shown in Figure 9 and Figure 10.

Case 1. In Case 2 a recursive call is made with an input that will not itself satisfy Case 1 or Case 2. Therefore the number of operations performed is $\mathcal{O}(m) = \mathcal{O}(n)$. See Figure 9 and Figure 10 for a concrete example of Algorithm 5.2.

6 Experimental Performance and Parallelism

A C implementation of each algorithm is available [3]. For each i in $4 \dots 7$, a uniform random sample of 100 plane triangulations of order 10^i was generated. Each algorithm was run on a each set of samples, see the timings in Figure 12.

Plane triangulations were sampled using the method of Poulalhon and Schaeffer [9]. The vertices were randomly re-labeled in each sampled graph, and the corresponding adjacency list arrays were re-ordered in memory such that the list for newly-labeled vertex 0 came first in memory, followed by the list for vertex 1, and so on. The benchmarks results therefore represent each algorithm's performance on a set of uniformly sampled plane triangulations, each with a uniformly sampled representation layout in memory. The list assignments for Algorithm 5.2 were uniformly drawn from a set of six colors.

A breadth-first search implemented with a circular array FIFO queue was run on the same set of graphs as a baseline algorithm that hits every half-edge of a graph exactly once. All binaries were compiled with Clang 19.1.7 and run on a Linux 6.12.9 machine with an x86_64 Intel N100 processor. The source code and binaries are available [3].

The performance of Algorithm 5.2 was similar to Algorithm 4.3 on the sampled graphs, but Algorithm 5.2 required an augmented adjacency list representation. For situations where such a representation is not immediately available, the time required by Algorithm 5.2 would also include the time required by Algorithm 3.1.

A benchmark was also run to test the asymptotic performance of Algorithm 4.2 and Algorithm 4.3 on the collection of pyramid graphs $\{A_k\}_{k \in \mathbb{N}}$ from Figure 3. Algorithm 4.2 scaled very poorly on pyramids, as expected; see the timings in Figure 13. Once again,

	$n = 10^3$	$n = 10^4$	$n = 10^5$	$n = 10^6$	$n = 10^7$
Breadth-first search	$9.72 \cdot 10^4$	$2.82 \cdot 10^5$	$3.94 \cdot 10^6$	$1.04 \cdot 10^8$	$1.31 \cdot 10^9$
Algorithm 3.1	$3.35 \cdot 10^4$	$5.34 \cdot 10^5$	$2.10 \cdot 10^7$	$4.15 \cdot 10^8$	$4.65 \cdot 10^9$
Algorithm 4.2	$2.24 \cdot 10^5$	$4.02 \cdot 10^6$	$6.78 \cdot 10^7$	$1.99 \cdot 10^9$	$3.57 \cdot 10^{10}$
Algorithm 4.3	$4.63 \cdot 10^4$	$5.99 \cdot 10^5$	$7.38 \cdot 10^6$	$1.58 \cdot 10^8$	$1.83 \cdot 10^9$
Algorithm 5.2	$4.59 \cdot 10^4$	$6.46 \cdot 10^5$	$9.89 \cdot 10^6$	$1.84 \cdot 10^8$	$2.12 \cdot 10^9$

Figure 12: Average time (ns) per triangulated plane graph (sample size 100).

vertex labels were uniformly shuffled to remove dependence on a particular representation. Values of k were selected such that $n(A_k) \approx 10^i$.

	$n \approx 10^3$	$n \approx 10^4$	$n \approx 10^5$	$n \approx 10^6$	$n \approx 10^7$
Algorithm 4.2	$2.59 \cdot 10^5$	$8.71 \cdot 10^6$	$4.78 \cdot 10^8$	$7.38 \cdot 10^{10}$	$2.95 \cdot 10^{12}$
Algorithm 4.3	$1.95 \cdot 10^4$	$2.07 \cdot 10^5$	$3.49 \cdot 10^6$	$1.48 \cdot 10^8$	$2.26 \cdot 10^9$

Figure 13: Average time (ns) per pyramid graph from Figure 3 (sample size 30).

Both Algorithm 4.3 and Algorithm 5.2 perform recursive calls that operate on subgraphs that are disjoint except for select vertices on their respective outer faces. Therefore it is possible to maintain a stack of independent recursive frames and have a pool of threads operate on the frames concurrently. The amount of parallelism possible with this method is highly dependent on the structure of the graph. E.g. when Algorithm 4.3 is run on a pyramid graph from Figure 3 there will never be more than one recursive frame on the stack, and thus no opportunity for parallel execution.

In the case of Algorithm 4.3 such parallelism can easily be organized since each frame writes only to vertices interior to the outer face of its associated subgraph. Therefore the implementation can maintain a shared lock-guarded stack from which all threads push and pop recursive frames. If no frames are available, a thread must idle until either a new frame is pushed to the stack by another thread or until all threads are idle.

To reduce lock contention, each thread should also maintain a small local stack of frames, and only access the shared stack when this local stack is full or empty. For the implementation and hardware used for benchmarks, the most optimal setup proved to be a local stack of 16 frames for each thread. If a local stack filled up, 8 frames were pushed to the shared stack. If a local stack emptied, 8 frames were popped from the shared stack.

Benchmarks in Figure 14 showed benefit to the parallel Algorithm 4.3 when $n \geq 10^4$. The same sample of plane triangulations was used as in Figure 12.

Algorithm 5.2 is trickier to adapt to parallel execution. The first difficulty to consider is that the recursive calls made in Algorithm 5.2 relied on a sequential execution order. If we store the mark and neighbor range ($S[v]$ and $N[v]$) for x , y , and z alongside each frame, then the recursive calls in Case 3.2.1 with $u_2 = x$, Case 3.2.2, Case 3.2.3.1, and

Threads	$n = 10^3$	$n = 10^4$	$n = 10^5$	$n = 10^6$	$n = 10^7$
1	$4.63 \cdot 10^4$	$5.99 \cdot 10^5$	$7.38 \cdot 10^6$	$1.58 \cdot 10^8$	$1.83 \cdot 10^9$
2	$4.04 \cdot 10^4$	$4.61 \cdot 10^5$	$4.49 \cdot 10^6$	$9.04 \cdot 10^7$	$1.04 \cdot 10^9$
3	$4.12 \cdot 10^4$	$3.58 \cdot 10^5$	$3.50 \cdot 10^6$	$6.75 \cdot 10^7$	$7.83 \cdot 10^8$
4	$4.32 \cdot 10^4$	$3.14 \cdot 10^5$	$2.98 \cdot 10^6$	$5.59 \cdot 10^7$	$6.53 \cdot 10^8$

Figure 14: Parallel Algorithm 4.3 average time (ns) per graph (sample size 100).

Case 3.2.3.2 with $u_2 = y$ become order independent. Unfortunately, in Case 3.2.1 with $u_2 \neq x$ and Case 3.2.3.2 with $u_2 \neq y$, the second recursive call will always depend on the reduced color list $L[u_2]$ produced by the first call.

The hurdle of dependent frames can be overcome by maintaining a shared lock-guarded object pool of frames and a stack of frame references. Recursive frames will be added to the pool when they appear in the usual course of the algorithm, but will only be pushed to the shared stack when they are ready to be executed. For each vertex $v \in G$, maintain an optional reference to a frame that will become ready when v has been colored, i.e. has had its color list reduced to length 1. To allow for multiple frames to await the coloring of a single vertex, each frame in the pool will store an “intrusive linked list,” that is, an optional reference to the next frame in line. When a frame is added to the pool and it needs to await a vertex that has another frame waiting on it, the new frame is added to the linked list of the existing frame. When a vertex v is colored, walk the linked list of frames waiting on v and push a reference to each frame onto the shared stack.

Another important detail is that the new unique face marks assigned to S must be unique across all threads, or else we risk corruption and data races against the array M . Uniqueness across threads can be ensured by using a shared atomic mark counter. Contention can be minimized by having threads reserve marks in chunks, rather than performing an atomic fetch-add operation for each new mark.

In Case 3.2.1, if $u_2 = x$, then $S[x]$ must be assigned a new unique mark in the recursive frame handling the G_2 subgraph to prevent future writes to $M[S[x]]$ from interfering with reads to $M[S[x]]$ regarding the vertices on the outer face of G_1 marked with $S[x]$. In Figure 9 the mark assigned to the bottom right vertex x of the graph in the step at the bottom right of the page would change from 4 to 7 in the G_2 block. This would then ripple forward, with the next new mark being an 8, and so on.

Lock contention can be reduced in a similar manner to Algorithm 4.3 by having each thread maintain a local stack of frames. Unfortunately, if a frame is created that needs to await the coloring of a vertex, then it must be added to the shared pool since any thread may color that vertex. Moreover, if a frame colors a vertex that has another frame waiting on it, then the lock must be acquired.

Benchmarks in Figure 15 showed some benefit to the parallel version of Algorithm 5.2 when $n \geq 10^4$, with significant performance improvements only appearing for very large graphs. The same sample of plane triangulations was used as in Figure 12.

Threads	$n = 10^3$	$n = 10^4$	$n = 10^5$	$n = 10^6$	$n = 10^7$
1	$4.59 \cdot 10^4$	$6.46 \cdot 10^5$	$9.89 \cdot 10^6$	$1.84 \cdot 10^8$	$2.12 \cdot 10^9$
2	$6.22 \cdot 10^4$	$5.99 \cdot 10^5$	$6.97 \cdot 10^6$	$1.23 \cdot 10^8$	$1.40 \cdot 10^9$
3	$6.09 \cdot 10^4$	$5.02 \cdot 10^5$	$5.65 \cdot 10^6$	$9.32 \cdot 10^7$	$1.06 \cdot 10^9$
4	$6.58 \cdot 10^4$	$4.82 \cdot 10^5$	$5.15 \cdot 10^6$	$7.89 \cdot 10^7$	$8.90 \cdot 10^8$

Figure 15: Parallel Algorithm 5.2 average time (ns) per graph (sample size 100).

References

- [1] J. Boyer and W. Myrvold, On the cutting edge: simplified $O(n)$ planarity by edge addition, *J. Graph Algorithms Appl.* **8** (2004), 241–273.
- [2] I. Broere and C. M. Mynhardt, Generalized colorings of outerplanar and planar graphs, *Graph theory with applications to algorithms and computer science* (Kalamazoo, Mich., 1984), pp. 151–161, Wiley-Intersci. Publ., Wiley, New York, 1985.
- [3] A. Bross, *Implementing path coloring algorithms on planar graphs*, Masters Project, University of Alaska, 2017, available from http://github.com/permutationlock/path_coloring_bgl and <http://github.com/permutationlock/libavengraph>.
- [4] G. G. Chappell and C. Hartman, Path choosability of planar graphs, *The Electronic Journal of Combinatorics* **25** (2018), P4.33.
- [5] G. Chartrand and H. V. Kronk, The point-arboricity of planar graphs, *J. London. Math. Soc.* **44** (1969), 612–616.
- [6] W. Goddard, Acyclic colorings of planar graphs, *Discrete Math.* **91** (1991), no. 1, 91–94.
- [7] C. M. Hartman, *Extremal Problems in Graph Theory*, Ph.D. Thesis, University of Illinois, 1997.
- [8] K. S. Poh, On the linear vertex-arboricity of a planar graph, *J. Graph Theory* **14** (1990), no. 1, 73–75.
- [9] D. Poulalhon and G. Schaeffer, Optimal Coding and Sampling of Triangulations, *Algorithmica*, **46** (2006), 505–527
- [10] R. Škrekovski, List improper colourings of planar graphs, *Combin. Probab. Comput.* **8** (1999), no. 3, 293–299.
- [11] D. B. West, *Introduction to Graph Theory*, 2nd ed., Prentice Hall, Upper Saddle River, NJ, 2000.